

如何寫出可以被看懂的程式

縮排

為什麼要縮排

你可能想說:「我程式可以執行就好了啊!」。但是你可以看看這個範例。

</> 縮排的錯誤示範

```
#include<bits/stdc++.h>
using namespace std;
int main(){
int n,a=0,sum=0;
cin>>n;
for(int i=0;i<n;++i){
cin>>a;
if(a<120)
sum+=a;
}
cout<<sum<<"\n";
}
```

請問你能在 20 秒內了解這個程式在幹嘛嗎? 如果可以, 那你就比筆者強大了^ ^。(至少在程式碼識讀上)

即便如此, 如果你遇到問題, 被你問到的人都絕對不會想要看到這樣的程式碼, 你如果傳這種給其他人, 大概有 99.99% 的機率被退回。

怎麼做比較好

程式撰寫風格是非常自由, 但一個大原則就是要有可讀性。我所使用的縮排原則是, 只要遇到函式(Function)、for, while 迴圈、if, switch 判斷式, 都會在接下來多空 2-4 個空格(在 vscode 裡面)。

以下是一個範例。

</> 縮排的合理示例

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int a = 0, sum = 0;
    cin >> n;
    for(int i = 0; i < n; ++i) {
        cin >> a;
        if(a < 120)
            sum += a;
    }
    cout << sum << "\n";
}
```

在這樣的階層式架構下, 只要是 APCS 觀念題有拿到 3, 4 級分的人應該都可以快速理解這段程式碼。

💡 Tip

程式碼撰寫上應以縮排的方式增加可讀性。在所有的程式碼中應用這樣的習慣，將有助於維護你寫的程式碼們。

空格

聲明

以下的建議僅供參考，空格的方式一直以來都是吵不完的議題，我將說明為何要這樣做。請依照你的個人需求選擇你的空格方式。(注意可讀性即可)

一般情況

因為會把程式碼的字體縮小，方便我閱讀更多行的程式碼，所以我會在運算符號間加上空格，這樣可以更容易懂。

</> 我的空格習慣(一般情況)

```
void solve() {  
    int a = 0, sum = 0;  
    cout << sum << "\n";  
}
```

</> 不加上空格的版本

```
void solve(){  
    int a=0,sum=0;  
    cout<<sum<<"\n";  
}
```

應該可以看出來我的程式碼比較壓縮,如果不喜歡這一種可以用第二種。基本上都可以被廣泛接受。

特殊情況

看到一般情況應該就會想到有特殊的情形。而為了可讀性，我在下列情形會加上空格。

1. 使用 if 而不想要換行時。
2. 把兩個指令塞在同一行時。
3. 運用指標時。
4. 運算式太複雜時。

以下程式是範例。

</> 特殊情況們

```
void solve(){  
    if(a<0) a=0;  
    b++; c++;  
    d->sum += e->sum;  
    cout<<a + (b-c) - d->sum<<"\n";  
}
```

不過盡可能避免這樣的特殊情況，因為這樣仍會讓程式碼變得難以閱讀。

💡 Tip

請依照你習慣的方式，有一致性與可讀性的加上空格。你會發現本章節強調可讀性，因為我實在是看過好多沒有辦法閱讀的程式，我還必須手動排版，造成每個人時間的浪費。

變數命名

這個章節我們要來探討變數名稱的意義。在業界，變數名稱要盡可能包含所有資訊，讓共事的夥伴可以藉由變數名稱讀懂程式碼。然而，有些人認為競賽只要用盡可能簡潔的變數名稱就好。

在簡短與明確中抉擇

我承認競賽上不太可能用向實務一樣長的變數名稱，因為競賽必須與時間賽跑。但太過簡短，甚至沒有意義的名稱可能導致除錯困難，因此會有一些折衷的方案，希望盡可能同時解決兩邊的問題。

縮寫

在命名變數時，使用英文縮寫讓程式碼變的簡短，是一個常見的方式。以下是一個簡單的表格，列舉出我可能會用到的縮寫代表的意義。

縮寫	ct	isv	mx	mn	idx	num
意義	count	is valid	max	min	index	number

組合型單字

對於組合型單字，我們可以使用兩種方式命名。

1. 加上底線，Ex: item_number。
2. 除了第一個單詞，後面的單詞第一個字母大寫，Ex: itemNumber

其他

其他的命名方式可以參考 Code Complete 第 11 章「變數名稱的力量」。

範例與練習

Example: 快速幂。

</> 排版範例

```
using ll=long long;
const ll MOD=1e9+7;

ll POW(ll a,ll x){
    ll ret=1;
    while(x>0){
        if(x%2==1) ret=ret*a%MOD;
        a=a*a%MOD;
        x/=2;
    }
    return ret;
}
```

Problem: 請對以下程式碼排版。

</> Code

```
#include<bits/stdc++.h>
using namespace std;

using ll=long long;
const ll INF=0x3f3f3f3f;
const ll LINF=0x3f3f3f3f3f3f3f3f;
const ll MOD=1e9+7;

int n;
int dp[20][(1<<16)+10];
int a[20];
int main(){
ios::sync_with_stdio(0);cin.tie(0);cin>>n;
for(int i=1;i<=n;++i) cin>>a[i];
for(int i=1;i<=n;++i) dp[i][0]=0;
for(int i=1;i<(1<<n);++i){
for(int k=1;k<=n;++k){
for(int l=1;l<=n;++l){
if( (i-(1<<l))^i == (1<<k) ){dp[k][i];
}}
}}
}
```

更進一步

可以去看這一部影片 <https://www.youtube.com/watch?v=CFRhGnuXG-4> , 重點是不要有太多層的迴圈。

Example: 五層的範例

</> 太多層的例子

```
bool check(int a,int b){
    if(a+b>100){
        if(a-b<50){
            for(int i=1;i<a;i*=2;++i){
                if(a*i%1000==200){
                    for(int j=0;j<a+b;++j){
                        a=(a+b)%100;
                        b=(a-b)%100;
                    }
                }
                return a%2;
            }
        }else{
            return a%2;
        }
    }else{
        return b%2;
    }
}
```

同樣的程式碼可以寫成這樣，會比較容易掌握你要表達的意義。

</> 改進的版本

```
void modify(int &a,int &b){
    for(int j=0;j<a+b;++j){
        a=(a+b)%100;
        b=(a-b)%100;
    }
}

bool check(int a,int b){
    if(a+b<=100){
        return b%2;
    }

    if(a-b>=50){
        return a%2;
    }

    for(int i=1;i<a;i*=2;++i){
        if(a*i%1000==200){
            modify(a,b);
            return a%2;
        }
    }
}
```

除此之外，我們也可以嘗試遵守一個原則，就是一個函數只做一件事情，並且不要超過 30 行，這樣會讓程式碼更容易閱讀。